

RULA: Risky Users Log Analyzer

Benjamin Fisher

Faculty Advisors: Paul Chauvet, Michael Curry

Project Objective

RULA is a lightweight CLI application that helps Information Security teams analyze Risky User logs in Microsoft Entra ID more efficiently by reducing time spent sorting through noisy entries and surfacing the important risks faster.

Research Questions

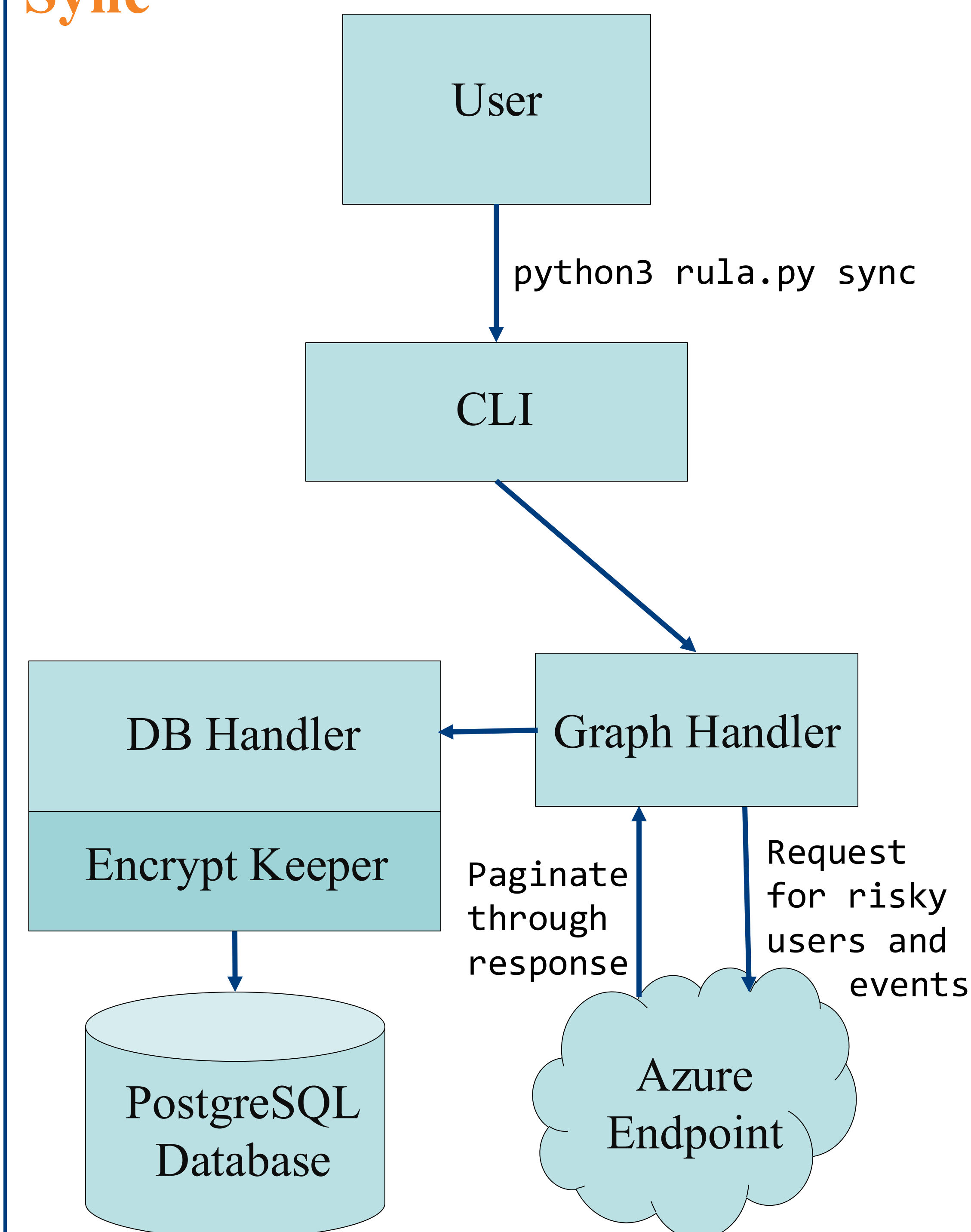
1. What steps can I take to reduce the amount of time spent investigating each risky user?
2. What risk patterns do I need to detect?
3. How do I assess and score user risk beyond Azure's native classifications?
4. How can I design a CLI tool that integrates with existing security workflows?

```
cuomor22@domain.edu [student]
Azure Risk: high (atRisk)
Custom Risk: high (score: 45)
Tags:

- anonymous_ip_usage [low]
  Details: {"anonymous_access_count": 3}

- azure_app_password_signins [critical]
  Details: {"app_names": ["Microsoft Azure CLI"],
"attempt_count": 6, "failed_signins": 6,
"time_window_hours": 24, "interrupted_signins": 0}
  Assessed: 2026-04-28 18:09:03.101272+00:00
```

Sync



```
deerej@domain.edu [faculty_staff]
Azure Risk: high (atRisk)
Custom Risk: critical (score: 70)
Tags:

- anonymous_ip_usage [low]
  Details: {"anonymous_access_count": 4}

- impossible_travel [high]
  Details: {"to_location": "New York, US", "from_location": "Go Vap, VN",
"time_difference_hours": 3.4}

- azure_app_password_signins [critical]
  Details: {"app_names": ["Azure Active Directory PowerShell",
"Azure Portal", "Windows Azure Active Directory"],
"attempt_count": 4, "failed_signins": 4,
"time_window_hours": 24, "interrupted_signins": 0}
  Assessed: 2026-04-20 14:48:44.814190+00:00
```

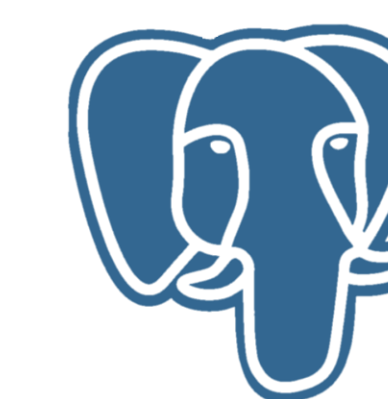
Setup and Use

- Register RULA in Azure and insert Tenant ID, Client ID, and client secret as environment variables into RULA
- Grant required API application permissions for RULA.
- VSCode “Reopen in Container” or build with docker-compose
- Sync users from Azure to local PostgreSQL DB with **sync**
- Build Assess risk using designated pattern analyzers with **assess-risk**
- Select users of desired customrisk score and output them to console or to CSV with **customrisk critical / high / medium / low**

Technologies



Requests
Python
Library for
HTTP requests



PostgreSQL
Open source
Relational
DBMS



Click
Python
Library for
CLI design



Entra ID
Microsoft's
cloud based
IAM service

RULA: Risky Users Log Analyzer

Benjamin Fisher

Faculty Advisors: Paul Chauvet, Michael Curry

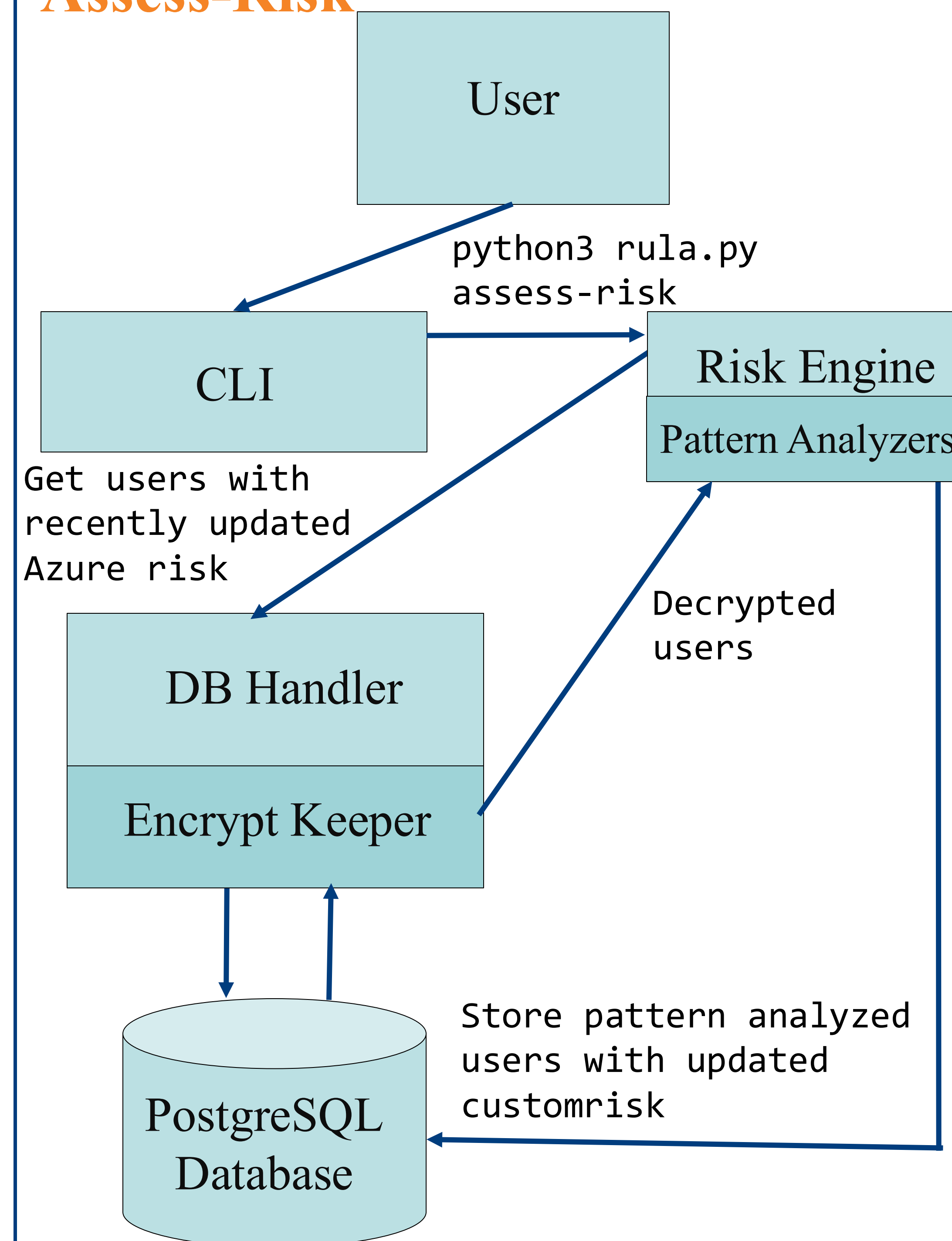
Project Objective

RULA is a lightweight CLI application that helps Information Security teams analyze Risky User logs in Microsoft Entra ID more efficiently by reducing time spent sorting through noisy entries and surfacing the important risks faster.

Research Questions

1. What steps can I take to reduce the amount of time spent investigating each risky user?
2. What risk patterns do I need to detect?
3. How do I assess and score user risk beyond Azure's native classifications?
4. How can I design a CLI tool that integrates with existing security workflows?

Assess-Risk



Setup and Use

- Register RULA in Azure and insert Tenant ID, Client ID, and client secret as environment variables into RULA
- Grant required API application permissions for RULA.
- VSCode “Reopen in Container” or build with docker-compose
- Sync users from Azure to local PostgreSQL DB with **sync**
- Build Assess risk using designated pattern analyzers with **assess-risk**
- Select users of desired customrisk score and output them to console or to CSV with **customrisk critical / high / medium / low**

`cuomor22@domain.edu` [student]

Azure Risk: high (atRisk)

Custom Risk: high (score: 45)

Tags:

- `anonymous_ip_usage` [low]

Details: {"anonymous_access_count": 3}

- `azure_app_password_signins` [critical]

Details: {"app_names": ["Microsoft Azure CLI"], "attempt_count": 6, "failed_signins": 6, "time_window_hours": 24, "interrupted_signins": 0}
Assessed: 2026-04-28 18:09:03.101272+00:00

`deerej@domain.edu` [faculty_staff]

Azure Risk: high (atRisk)

Custom Risk: critical (score: 70)

Tags:

- `anonymous_ip_usage` [low]

Details: {"anonymous_access_count": 4}

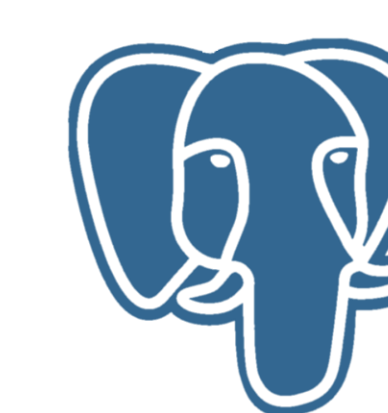
- `impossible_travel` [high] Details: {"to_location": "New York, US", "from_location": "Go Vap, VN", "time_difference_hours": 3.4}

- `azure_app_password_signins` [critical] Details: {"app_names": ["Azure Active Directory PowerShell", "Azure Portal", "Windows Azure Active Directory"], "attempt_count": 4, "failed_signins": 4, "time_window_hours": 24, "interrupted_signins": 0}
Assessed: 2026-04-20 14:48:44.814190+00:00

Technologies



Requests
Python
Library for
HTTP requests



PostgreSQL
Open source
Relational
DBMS



Click
Python
Library for
CLI design



Entra ID
Microsoft's
cloud based
IAM service

RULA: Risky Users Log Analyzer

Benjamin Fisher

Faculty Advisors: Paul Chauvet, Michael Curry

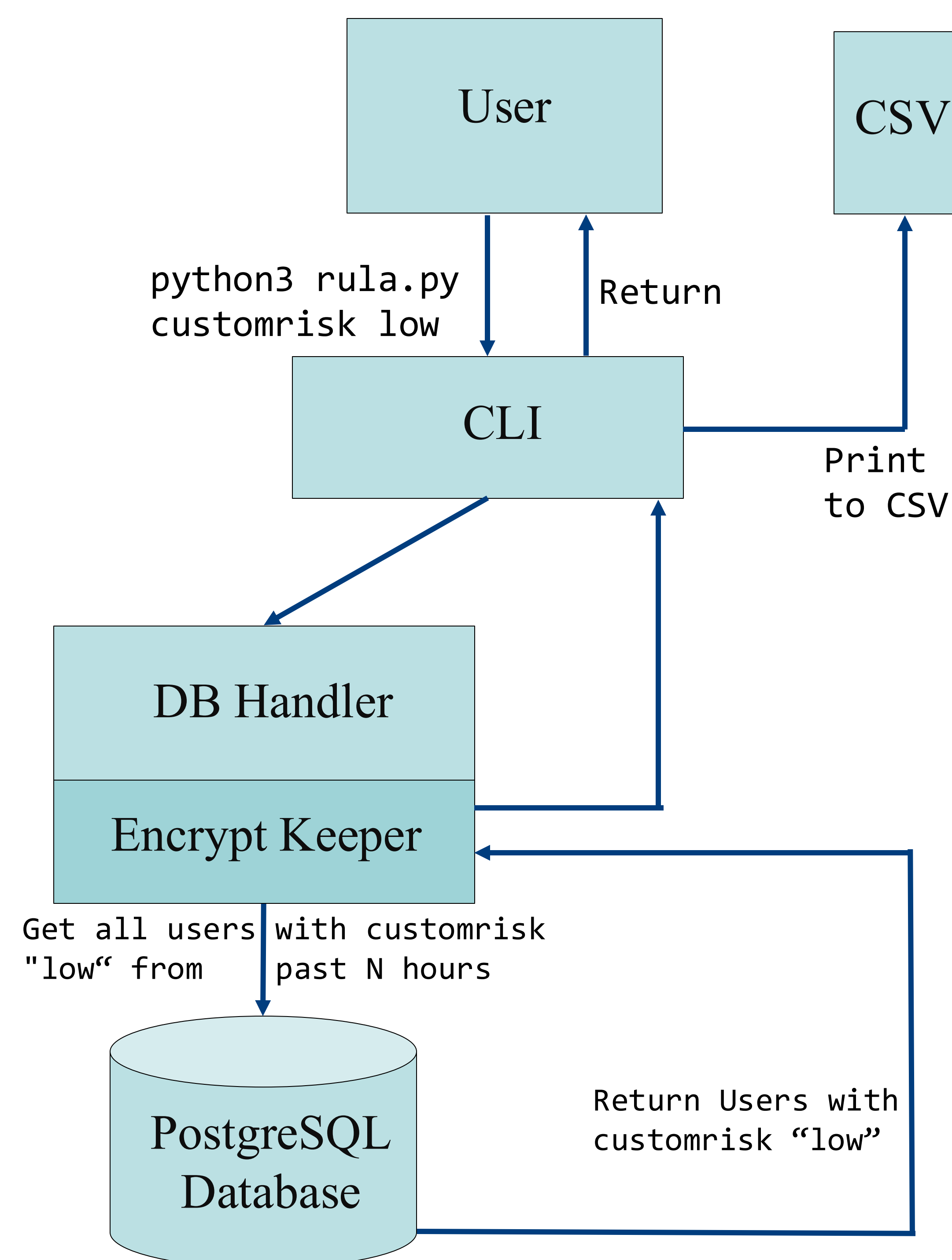
Project Objective

RULA is a lightweight CLI application that helps Information Security teams analyze Risky User logs in Microsoft Entra ID more efficiently by reducing time spent sorting through noisy entries and surfacing the important risks faster.

Research Questions

1. What steps can I take to reduce the amount of time spent investigating each risky user?
2. What risk patterns do I need to detect?
3. How do I assess and score user risk beyond Azure's native classifications?
4. How can I design a CLI tool that integrates with existing security workflows?

Customrisk



Setup and Use

- Register RULA in Azure and insert Tenant ID, Client ID, and client secret as environment variables into RULA
- Grant required API application permissions for RULA.
- VSCode “Reopen in Container” or build with docker-compose
- Sync users from Azure to local PostgreSQL DB with **sync**
- Build Assess risk using designated pattern analyzers with **assess-risk**
- Select users of desired customrisk score and output them to console or to CSV with **customrisk critical / high / medium / low**

`cuomor22@domain.edu` [student]

Azure Risk: high (atRisk)

Custom Risk: high (score: 45)

Tags:

- `anonymous_ip_usage` [low]

Details: {"anonymous_access_count": 3}

- `azure_app_password_signins` [critical]

Details: {"app_names": ["Microsoft Azure CLI"], "attempt_count": 6, "failed_signins": 6, "time_window_hours": 24, "interrupted_signins": 0}
Assessed: 2026-04-28 18:09:03.101272+00:00

`deerej@domain.edu` [faculty_staff]

Azure Risk: high (atRisk)

Custom Risk: critical (score: 70)

Tags:

- `anonymous_ip_usage` [low]

Details: {"anonymous_access_count": 4}

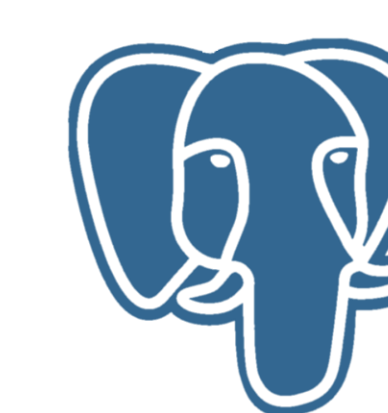
- `impossible_travel` [high] Details: {"to_location": "New York, US", "from_location": "Go Vap, VN", "time_difference_hours": 3.4}

- `azure_app_password_signins` [critical] Details: {"app_names": ["Azure Active Directory PowerShell", "Azure Portal", "Windows Azure Active Directory"], "attempt_count": 4, "failed_signins": 4, "time_window_hours": 24, "interrupted_signins": 0}
Assessed: 2026-04-20 14:48:44.814190+00:00

Technologies



Requests
Python
Library for
HTTP requests



PostgreSQL
Open source
Relational
DBMS



Click
Python
Library for
CLI design



Entra ID
Microsoft's
cloud based
IAM service